데이터 흐름 (Data Flow)

개요

이 문서는 EVM 실행부터 영지식 증명 생성까지의 전체 데이터 흐름을 설명합니다. 데이터는 여러 단계의 변환을 거쳐 최종적으로 증명으로 변환됩니다.

전체 데이터 흐름 다이어그램

```
┌─────────────────────────────────────────────────────────────┐
│ 1. EVM Execution (Geth) │
│ │
│ Transaction → EVM → GethExecTrace │
│ ├─ struct_logs: Vec<GethExecStep> │
│ ├─ gas, failed, return_value │
│ └─ account_after │
└─────────────────────────┬───────────────────────────────────┘
│
↓
┌─────────────────────────────────────────────────────────────┐
│ 2. Bus-Mapping (CircuitInputBuilder) │
│ │
```

```
| GethExecTrace → CircuitInputBuilder → Block |
| ├─ handle_block() |
| ├─ handle_tx() |
| └─ gen_associated_ops() |
| |
| 생성되는 데이터: |
| ├─ ExecStep: 각 실행 단계 |
| ├─ RwOp: Read-Write 연산 |
| ├─ Transaction: 트랜잭션 데이터 |
| └─ Call: 호출 컨텍스트 |
└──────────────────────────┬──────────────────────────────────────┘
|
↓
┌──────────────────────────────────────────────────────────────────┐
| 3. Witness Generation (block_convert) |
| |
| Block (bus-mapping) → Block (witness) |
| ├─ Bytecode |
| ├─ Transaction |
| ├─ ExecStep |
| ├─ RwMap |
| └─ MptUpdates |
└──────────────────────────┬──────────────────────────────────────┘
```

```
│
↓
┌──────────────────────────────────────────────────────────────┐
│ 4. Circuit Witness Assignment │
│ │
│ Block → Circuit.new_from_block() → Circuit Instance │
│ ├─ EvmCircuit │
│ ├─ StateCircuit │
│ ├─ TxCircuit │
│ ├─ BytecodeCircuit │
│ └─ ... (기타 회로들) │
└──────────────────────────┬───────────────────────────────────┘
│
↓
┌──────────────────────────────────────────────────────────────┐
│ 5. Proof Generation │
│ │
│ Circuit Instance → Halo2 Prover → Proof │
│ ├─ Setup │
│ ├─ Synthesize │
│ └─ Prove │
└──────────────────────────────────────────────────────────────┘
```

단계별 상세 설명

1단계: EVM 실행 및 트레이스 수집

입력: 이더리움 트랜잭션

출력: `GethExecTrace`

GethExecTrace 구조

```
// eth-types/src/geth_types.rs
pub struct GethExecTrace {
pub struct_logs: Vec<GethExecStep>, // 각 opcode 실행 단계
pub gas: Gas, // 사용된 총 gas
pub failed: bool, // 실행 실패 여부
pub return_value: String, // 반환값
pub account_after: Vec<AccountTrace>, // 실행 후 계정 상태
pub l1_fee: u64, // L1 수수료 (Scroll)
}

pub struct GethExecStep {
pub pc: ProgramCounter, // Program Counter
pub op: OpcodeId, // Opcode
pub gas: Gas, // 남은 gas
pub gas_cost: GasCost, // 이 단계의 gas 비용
pub depth: u16, // 호출 깊이
```

```
pub stack: Option<Vec<Word>>, // 스택 상태
pub memory: Option<Vec<Word>>, // 메모리 상태
pub storage: Option<HashMap<Word, Word>>, // 스토리지 상태
pub refund: Gas, // 환불 gas
// ...
}
```

수집 과정:

1. Geth 클라이언트가 트랜잭션 실행
2. 커스텀 트레이서가 각 opcode 실행 시점에 상태 기록
3. 실행 완료 후 전체 트레이스 반환

2단계: Bus-Mapping - 트레이스를 회로 입력으로 변환

입력: `GethExecTrace`

출력: `Block` (bus-mapping 버전)

CircuitInputBuilder의 역할

`CircuitInputBuilder` 는 트레이스를 회로가 이해할 수 있는 형태로 변환하는 핵심 구조체입니다:

```
// bus-mapping/src/circuit_input_builder.rs
pub struct CircuitInputBuilder {
pub sdb: StateDB, // 상태 데이터베이스
pub code_db: CodeDB, // 코드 데이터베이스
pub block: Blocks, // 블록 데이터 (청크)
pub block_ctx: BlockContext, // 블록 컨텍스트
```

```
}
```

처리 흐름

```
// 1. 블록 처리 시작
impl CircuitInputBuilder {
pub fn handle_block(
&mut self,
eth_block: &EthBlock,
geth_traces: &[GethExecTrace],
) -> Result<(), Error> {
// 각 트랜잭션 처리
for (tx_index, tx) in eth_block.transactions.iter().enumerate() {
let geth_trace = &geth_traces[tx_index];
self.handle_tx(tx, geth_trace)?;
}
// 최종화
self.finalize_building()?;
Ok(())
}
}


// 2. 트랜잭션 처리
```

```rust
fn handle_tx(
&mut self,
eth_tx: &Transaction,
geth_trace: &GethExecTrace,
) -> Result<(), Error> {
// 트랜잭션 생성
let mut tx = self.new_tx(eth_tx, !geth_trace.failed)?;
let mut tx_ctx = TransactionContext::new(eth_tx, geth_trace)?;
// BeginTx 단계 생성
let begin_tx_steps = gen_associated_steps(
&mut self.state_ref(&mut tx, &mut tx_ctx),
ExecState::BeginTx,
)?;
tx.steps_mut().extend(begin_tx_steps);
// 각 opcode 처리
for (index, geth_step) in geth_trace.struct_logs.iter().enumerate() {
let exec_steps = gen_associated_ops(
&geth_step.op,
&mut self.state_ref(&mut tx, &mut tx_ctx),
&geth_trace.struct_logs[index..],
)?;
tx.steps_mut().extend(exec_steps);
}
```

```
// EndTx 단계 생성
let end_tx_steps = gen_associated_steps(
&mut self.state_ref(&mut tx, &mut tx_ctx),
ExecState::EndTx,
)?;
tx.steps_mut().extend(end_tx_steps);
self.block.txs.push(tx);
Ok(())
}
```

생성되는 데이터 구조

ExecStep: 각 실행 단계를 표현

```
// bus-mapping/src/execution.rs
pub struct ExecStep {
pub exec_state: ExecState, // 실행 상태 (Opcode 또는 특수 상태)
pub gas_left: Gas, // 남은 gas
pub gas_cost: GasCost, // 이 단계의 gas 비용
pub pc: ProgramCounter, // Program Counter
pub stack_size: usize, // 스택 크기
pub memory_size: u64, // 메모리 크기
pub rwc: RWCounter, // RW 카운터
pub bus_mapping_instance: Vec<OperationRef>, // RW 연산 참조
```

```
// ...
}
```

RwOp: Read-Write 연산

```
// bus-mapping/src/operation.rs
pub enum RwOp {
Stack { /* ... */ },
Memory { /* ... */ },
Storage { /* ... */ },
TxAccessListAccount { /* ... */ },
TxAccessListAccountStorage { /* ... */ },
TxRefund { /* ... */ },
Account { /* ... */ },
CallContext { /* ... */ },
TxReceipt { /* ... */ },
TxLog { /* ... */ },
}
```

Opcode별 처리 예시

```
// bus-mapping/src/evm/opcodes/add.rs
// ADD opcode 처리
pub fn gen_add_steps(
```

```
state: &mut CircuitInputStateRef,
geth_steps: &[GethExecStep],
) -> Result<Vec<ExecStep>, Error> {
let geth_step = &geth_steps[0];
let mut exec_step = state.new_step(geth_step)?;
// 스택에서 두 값 읽기
let a = state.stack_read(&mut exec_step, geth_step.stack.nth_last(0)?)?;
let b = state.stack_read(&mut exec_step, geth_step.stack.nth_last(1)?)?;
// 결과를 스택에 쓰기
let result = a.overflowing_add(b).0;
state.stack_write(&mut exec_step, geth_step.stack.nth_last(1)?, result)?;
Ok(vec![exec_step])
}
```

3단계: Witness 생성

입력: `Block` (bus-mapping)

출력: `Block` (witness)

block_convert 함수

```
// zkevm-circuits/src/witness/block.rs
pub fn block_convert(
block: &bus_mapping::circuit_input_builder::Block,
code_db: &bus_mapping::state_db::CodeDB,
```

```
) -> Result<Block, Error> {
// 1. 기본 블록 정보 복사
let mut witness_block = Block {
chain_id: block.chain_id,
history_hashes: block.history_hashes.clone(),
// ...
};
// 2. 트랜잭션 변환
for tx in &block.txs {
let witness_tx = convert_tx(tx)?;
witness_block.txs.push(witness_tx);
}
// 3. 바이트코드 변환
for (code_hash, bytecode) in code_db.0.iter() {
let witness_bytecode = convert_bytecode(bytecode)?;
witness_block.bytecodes.insert(*code_hash, witness_bytecode);
}
// 4. RW 연산 변환
witness_block.rws = convert_rw_map(&block.container)?;
// 5. MPT 업데이트 생성
witness_block.mpt_updates = generate_mpt_updates(block)?;
Ok(witness_block)
```

```
}
```

Witness Block 구조

```
// zkevm-circuits/src/witness/block.rs
pub struct Block {
pub chain_id: u64,
pub history_hashes: Vec<Word>,
pub eth_block: eth_types::Block<eth_types::Transaction>,
// 트랜잭션 데이터
pub txs: Vec<Transaction>,
// 바이트코드
pub bytecodes: HashMap<Word, Bytecode>,
// RW 연산
pub rws: RwMap,
// MPT 업데이트
pub mpt_updates: MptUpdates,
// 회로 파라미터
pub circuits_params: CircuitsParams,
// ...
}
```

4단계: 회로 Witness 할당

입력: `Block` (witness)

출력: 각 회로의 인스턴스

SuperCircuit 생성

```
// zkevm-circuits/src/super_circuit.rs
impl SubCircuit<Fr> for SuperCircuit<...> {
fn  new_from_block(block: &Block) -> Self {
// 각 회로 생성
let  evm_circuit = EvmCircuit::new_from_block(block);
let  state_circuit = StateCircuit::new_from_block(block);
let  tx_circuit = TxCircuit::new_from_block(block);
let  pi_circuit = PiCircuit::new_from_block(block);
let  bytecode_circuit = BytecodeCircuit::new_from_block(block);
let  copy_circuit = CopyCircuit::new_from_block_no_external(block);
let  exp_circuit = ExpCircuit::new_from_block(block);
let  keccak_circuit = KeccakCircuit::new_from_block(block);
let  sha256_circuit = SHA256Circuit::new_from_block(block);
let  poseidon_circuit = PoseidonCircuit::new_from_block(block);
let  rlp_circuit = RlpCircuit::new_from_block(block);
let  sig_circuit = SigCircuit::new_from_block(block);
let  modexp_circuit = ModExpCircuit::new_from_block(block);
let  ecc_circuit = EccCircuit::new_from_block(block);
let  mpt_circuit = MptCircuit::new_from_block(block);
```

```
SuperCircuit {
evm_circuit,
state_circuit,
tx_circuit,
// ... 기타 회로들
}
}
}
```

개별 회로의 Witness 할당 예시

```
// EVM Circuit
impl SubCircuit<Fr> for EvmCircuit<Fr> {
fn new_from_block(block: &Block) -> Self {
let mut circuit = Self::default();
// 각 트랜잭션의 각 단계 처리
for tx in &block.txs {
for step in tx.steps() {
// ExecStep을 회로 witness로 변환
circuit.assign_exec_step(step);
}
}
circuit
}
```

```
}
```

5단계: 증명 생성

입력: Circuit Instance

출력: Proof

증명 생성 흐름

```
// zkevm-circuits/src/super_circuit.rs
impl SuperCircuit<...> {
pub fn build(
geth_data: GethData,
circuits_params: CircuitsParams,
) -> Result<(u32, Self, Vec<Vec<Fr>>), Error> {
// 1. BlockData 생성
let block_data = BlockData::new_from_geth_data_with_params(
geth_data.clone(),
circuits_params,
);
// 2. CircuitInputBuilder 생성 및 실행
let mut builder = block_data.new_circuit_input_builder();
builder.handle_block(
&geth_data.eth_block,
```

```
&geth_data.geth_traces,
)?;
// 3. Witness Block 생성
let block = block_convert(&builder.block, &builder.code_db)?;
// 4. SuperCircuit 생성
let circuit = SuperCircuit::new_from_block(&block);
// 5. 필요한 k 계산
let (_, rows_needed) = Self::min_num_rows_block(&block);
let k = log2_ceil(Self::unusable_rows() + rows_needed);
// 6. Public Input 생성
let instance = circuit.instance();
Ok((k, circuit, instance))
}
}
```

데이터 변환 예시: ADD Opcode

전체 흐름을 ADD opcode를 예로 들어 설명합니다:

1. Geth 트레이스

```
{
"pc": 10,
"op": "ADD",
```

```
"gas": 1000,
"gasCost": 3,
"depth": 1,
"stack": ["0x5", "0x3"],
"memory": [],
"storage": {}
}
```

2. Bus-Mapping

```
ExecStep {
exec_state: ExecState::Op(OpcodeId::ADD),
gas_left: Gas(1000),
gas_cost: GasCost(3),
pc: ProgramCounter(10),
bus_mapping_instance: vec![
// Stack read: position 0, value 0x5
OperationRef(Stack { rw: Read, stack_pointer: 1023, value: 0x5 }),
// Stack read: position 1, value 0x3
OperationRef(Stack { rw: Read, stack_pointer: 1024, value: 0x3 }),
// Stack write: position 1, value 0x8
OperationRef(Stack { rw: Write, stack_pointer: 1024, value: 0x8 }),
],
```

```
}
```

3. Witness

```
// EVM Circuit witness
Row {
q_step: 1,
opcode: ADD,
stack_pointer: 1024,
gas_left: 1000,
// Stack 연산 witness
stack_read_0: 0x5,
stack_read_1: 0x3,
stack_write: 0x8,
// 제약 조건: stack_write == stack_read_0 + stack_read_1
}
```

4. 제약 조건

```
// EVM Circuit에서 ADD 제약 조건
meta.create_gate("ADD", |meta| {
let  q_step = meta.query_selector(q_step);
let  is_add = meta.query_advice(is_add, Rotation::cur());
```

```
let a = meta.query_advice(stack_read_0, Rotation::cur());
let b = meta.query_advice(stack_read_1, Rotation::cur());
let c = meta.query_advice(stack_write, Rotation::cur());
vec![
q_step * is_add * (a + b - c) // c == a + b
]
});
```

CircuitInputBuilder의 상태 관리

`CircuitInputBuilder` 는 실행 중 다양한 상태를 추적합니다:

```
pub struct BlockContext {
pub rwc: RWCounter, // 전역 RW 카운터
pub call_map: HashMap<usize, (usize, usize)>, // call_id → (tx_idx, call_idx)
pub cumulative_gas_used: u64, // 누적 gas 사용량
}

pub struct TransactionContext {
pub call_is_success: Vec<bool>, // 각 호출의 성공 여부
pub call_is_success_offset: usize, // 오프셋
// ...
}
```

최적화 및 특수 처리

RW Counter 관리

모든 Read-Write 연산은 전역 카운터로 순서가 매겨집니다:

```
pub struct RWCounter(pub usize);

impl RWCounter {
pub fn inc(&mut self) {
self.0 += 1;
}
}
```

메모리 최적화

큰 메모리 영역은 효율적으로 처리됩니다:

- 실제 사용된 영역만 추적
- Copy Circuit으로 대량 복사 처리

특수 상태 처리

- `BeginTx` : 트랜잭션 시작 시 초기화
- `EndTx` : 트랜잭션 종료 시 정리
- `EndBlock` : 블록 종료 시 최종화

관련 문서

이전/다음

- **이전**: 크레이트 구조 - 각 크레이트의 역할
- **다음**: EVM Circuit - EVM 실행 검증

같은 섹션

- 전체 구조 - 시스템 아키텍처 개요
- 크레이트 구조 - 크레이트 의존성

데이터 변환 과정

- Bus-Mapping 개요 - 트레이스 변환 시스템
- 트레이스 수집 - Geth 트레이서 연동
- RW 연산 - Read-Write 연산 생성
- Opcode 처리 - Opcode별 처리

Witness 생성

- Witness 개요 - Witness 생성 프로세스
- Block Witness - 블록 데이터
- Tx Witness - 트랜잭션 데이터
- Step Witness - 실행 단계 데이터

회로 구현

- EVM Circuit - EVM 실행 검증
- State Circuit - 상태 관리
- 테이블 시스템 - 회로 간 데이터 공유

참고 자료

- `bus-mapping/src/circuit_input_builder.rs` : CircuitInputBuilder 구현
- `zkevm-circuits/src/witness/block.rs` : Witness Block 구조
- `zkevm-circuits/src/super_circuit.rs` : SuperCircuit 통합
- `bus-mapping/src/evm/opcodes/` : Opcode별 처리 로직